Project_IoT — Tài liệu tổng hợp

Nov 5, 2026 · @Duong

1. Tổng quan

Dự án là một cánh tay robot 4 servo do người bấm phím điều khiển; 2 camera nhìn tờ A4, tính góc 3 servo cần tới và chỉ cho người bấm **phím gì, mấy lần**. Khi cả 3 servo đã tới đúng góc, màn hình báo **XONG**.

- **Đầu vào**: tờ A4 có 4 mã ArUco ở 4 góc, một vật có màu (cốc đỏ) đặt trên giấy.
- **Xử lý**: tìm vật → tọa độ (x, y) cm trên giấy → tính góc đế, vai, khuỷu (IK) → so với góc servo thật Arduino báo về.
- **Đầu ra**: lệnh bấm phím (ví dụ `D` × 3 = xoay đế 6°) và kết luận thành công.
- **Không tự gửi phím**: chế độ tự bấm (Auto-Step) đã bỏ khi đơn giản hóa ngày 2026-11-02; người dùng tự bấm.

Thành công = cả 3 servo (đế, vai, khuỷu) lệch không quá 1 bước bấm (2°) so với góc gợi ý, giữ liên tục 1 giây thì đếm thêm 1 lần. Kẹp không tính.

Code chia làm hai nhóm, nói chuyện với nhau qua một đối tượng `Plan`:

	Nhóm 1 — Điều khiển (`control.py`)	Nhóm 2 — Success (`success.py`)
Việc	tính góc cần tới, lập lệnh bấm phím, gửi phím qua USB	kết luận đã tới chưa, đếm số lần thành công
Đầu vào	các vật camera thấy + góc servo thật	`Plan` của nhóm 1
Đầu ra	`Plan` : góc hiện tại, góc cần tới, 3 `Move`	`Result` : `done` , từng servo đúng chưa, còn mấy lần bấm

Có hai máy tính: **Mac** đặt cạnh cánh tay (giữ 2 camera, chạy `main.py` , cắm USB vào Arduino) và **Dell** cách khoảng 2 m (xem màn hình Mac qua Wi-Fi, gửi phím qua Bluetooth). Chi tiết ở mục 6.

2. Phần cứng và sơ đồ mạch

Mạch gồm Arduino Uno + PCA9685 điều khiển 4 servo MG996R, nguồn servo riêng 5–6 V, HC-05 nhận phím Bluetooth và 2 camera cắm vào Mac.

Thành phần	Thông số	Ghi chú
Arduino Uno	board `arduino:avr:uno`, USB Serial 9600 baud	nhận phím từ USB và Bluetooth, gửi góc servo dạng JSON về Mac
PCA9685	I²C 0x40, xung 50 Hz	kênh 0 đế, 1 vai, 2 khuỷu, 3 kẹp
4 × servo MG996R	đế 0–180°, vai 25–155°, khuỷu 25–155°, kẹp 95–160°	Home = 90 / 90 / 90 / 95°
Nguồn tổ ong 5–6 V	vào cổng xanh (V+) của PCA9685	chỉ cấp cho servo, **không** lấy từ USB
HC-05	SoftwareSerial D2 / D3, 9600 baud	nhận phím từ máy Dell; mỗi lúc chỉ 1 kết nối
Cam TRÊN	OBSBOT Meet SE, 1920×1080	treo cao khoảng 46 cm, nhìn thẳng xuống tờ giấy
Cam NGANG	camera tích hợp MacBook, 1920×1080	đặt thấp trước tờ giấy; có thể thay bằng điện thoại
Tờ A4	29,7 × 21,0 cm, 4 mã ArUco `DICT_4X4_50` ID 0–3	in đúng 100%: cạnh mỗi mã phải đo được 2,8 cm

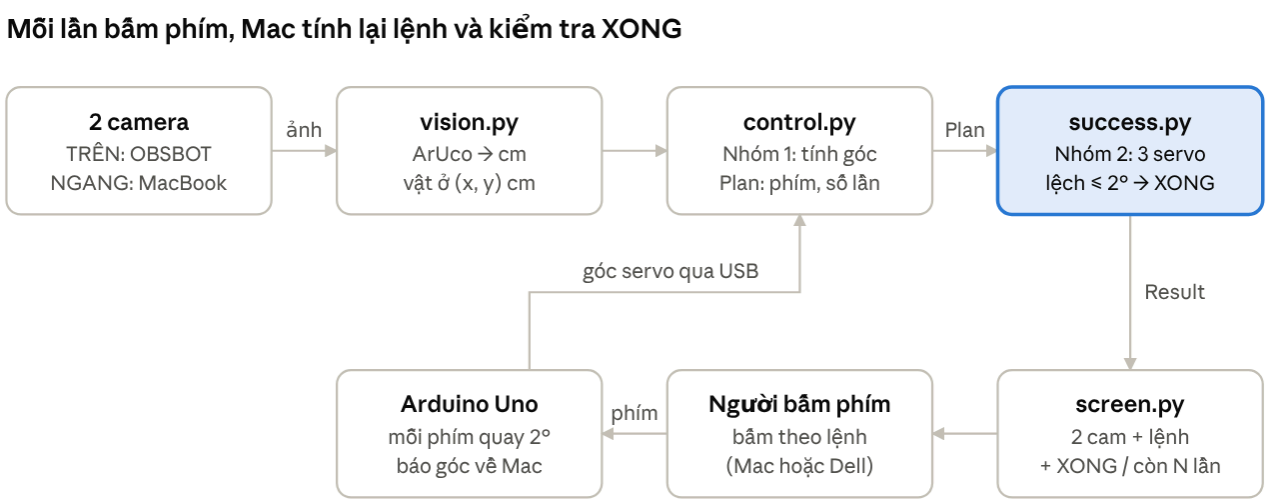
Dây nối (theo firmware và ghi chú slide):

- **Mac ↔ Arduino**: cáp USB, vừa truyền dữ liệu (phím xuống, JSON lên) vừa cấp nguồn cho Arduino.
- **Arduino → PCA9685**: 4 dây 5 V, GND, SDA (A4), SCL (A5).
- **PCA9685 → servo**: kênh 0–3, mỗi servo 3 dây (xung, V+, GND).
- **HC-05 → Arduino**: TXD vào D2, RXD vào D3.
- **GND chung** giữa Arduino, PCA9685 và nguồn servo, để xung và nguồn có cùng mức tham chiếu.

Câu hỏi mở: chân RXD của HC-05 có dùng cầu chia áp không? Ghi chú slide vẫn để trống, cần điền theo mạch thật.

3. Phần mềm: các tệp mã và luồng chạy

Mỗi khung hình trong `main.py` chỉ là ba dòng (dòng 194–197): `plan = control.update(tracker.objects)` → `result = success.update(plan)` → `build_screen(..., plan, result)`. Phần Python gồm 10 module, khoảng 3 380 dòng.



luồng mỗi khung hình · 7 khối, 1 vòng lặp

Vòng lặp khép qua Arduino: mỗi phím làm góc servo đổi, Arduino báo góc mới về, nhóm 1 tính lại lệnh và nhóm 2 xét lại XONG.

Tệp	Dòng	Vai trò
`main.py`	363	nối camera → vision → control → success → màn hình; phím, chuột, tham số dòng lệnh
`control.py`	620	**Nhóm 1**: hình học cánh tay (FK / IK), cổng USB Arduino, chọn vật, `Plan` + `Move`
`success.py`	75	**Nhóm 2**: `check(plan)` → `Result`; đếm số lần thành công
`screen.py`	271	vẽ màn hình: 2 camera + phần 1 + phần 2
`vision.py`	691	dò tờ A4 qua ArUco, tìm vật theo màu (YOLO tùy chọn), hiệu chỉnh phối cảnh
`cameras.py`	415	tìm camera theo tên, gán vai trò TRÊN / NGANG, luồng đọc nền tự mở lại
`calibration.py`	140	lưu / nạp hiệu chỉnh vào `calibration.json`
`arm_fit.py`	141	khớp hình học cánh tay từ các lần "ôm" cốc thật (Levenberg–Marquardt, chỉ numpy)
`phone.py`	341	chia sẻ màn hình qua Wi-Fi cho máy Dell; trang web dự phòng cho điện thoại

Tệp	Dòng	Vai trò
`remote.py`	321	app máy Dell: xem màn hình Mac, gửi phím qua Bluetooth
`generate_aruco_a4.py`	225	tạo tờ A4 có 4 mã ArUco để in
`robot_arm_4dof/robot_arm_4dof.ino`	208	firmware Arduino; nạp bằng `flash.sh`
`tests/`	2 143	13 tệp, 124 test

Hai nhóm tách biệt: nhóm 1 không biết thế nào là "xong", nhóm 2 không biết gì về camera, IK hay cổng USB. Cùng một `Plan` luôn cho cùng một kết luận.

4. Thị giác máy tính

Vị trí vật được đo bằng cm trên **tờ giấy**, không phụ thuộc camera nào nhìn: 4 mã ArUco cho mỗi camera một phép biến đổi Homography giữa pixel và cm.

- **Hệ tọa độ**: gốc ở góc ngoài mã ID 0; trục X chạy ID 0 → ID 1 (29,7 cm), trục Y chạy ID 0 → ID 3 (21,0 cm).
- **Che mất mã**: cánh tay che 1–2 mã vẫn chạy nhờ vị trí đã nhớ; camera bị xô thì tự quên; phím `U` quên thủ công.
- **Tìm vật (mặc định)**: tách màu HSV trên ảnh phẳng của tờ giấy (độ bão hòa ≥ 90, độ sáng ≥ 60). Bỏ viền 1 cm sát mép, khối quá to, khối chạm mép và khối dính phần đen của cánh tay.
- **YOLO11s**: chỉ bật khi chạy `--yolo`. Dùng trọng số COCO có sẵn, chưa fine-tune, hay gọi nhầm bộ phận robot (thân servo đen thành `traffic light`).
- **Chọn vật**: vật gần đế nhất được chọn trước, `TAB` đổi vật. Vị trí lấy trung bình 10 khung; vật bị che thì nhớ 120 giây.
- **Hiệu chỉnh phối cảnh**: cam TRÊN treo cao khoảng 46 cm nên nắp cốc cao 5 cm hiện lệch khoảng 1 cm so với đáy. Một phép đo ngày 02/11: hai camera lệch nhau 1,2 cm, sau hiệu chỉnh còn 0,2 cm.

$$p_{\text{thật}} = n + (p_{\text{đo}} - n)\cdot\frac{H - h}{H}$$

(n: tâm ảnh; H = 46 cm: độ cao camera, ước từ ảnh chứ không đo bằng thước; h: độ cao vật.)

Nhận đúng camera trên macOS: chỉ số `cv2.VideoCapture(i)` sắp theo `uniqueID`, còn danh sách của hệ điều hành đổi thứ tự giữa các lần chạy. `cameras.py` tìm camera theo **tên** rồi đổi sang chỉ số ngay trước mỗi lần mở. Phòng hờ, chương trình so độ xiên của tờ giấy trong 2 ảnh và cảnh báo nếu gán nhầm; phím `T` tráo vai trò.

5. Điều khiển và firmware

Mỗi phím quay đế, vai hoặc khuỷu **2°**, kẹp **5°**; Arduino báo góc 4 servo về Mac ngay sau mỗi phím và cứ 200 ms một lần.

Phím	Tác dụng
`A` / `D`	xoay đế sang **phải** / **trái** (nhìn theo cam NGANG), mỗi lần 2°
`W` / `S`	nâng / hạ vai, mỗi lần 2°
`I` / `K`	khuỷu lên / xuống, mỗi lần 2°
`J` / `L`	mở / khép kẹp, mỗi lần 5° (không tính vào thành công)
`H`	cả 4 servo về Home 90 / 90 / 90 / 95°
`X` / `Z`	ngắt xung servo kẹp / cả 4 servo để nguội
`TAB`	đổi vật mục tiêu
`B`	hiệu chỉnh vị trí đế robot (click vào trục xoay trên hình cam TRÊN)
`U` / `T` / `C` / `Q`	quên vị trí mã ArUco / tráo 2 camera / chụp ảnh vào `captures/` / thoát

Lưu ý: chú thích trong firmware ghi `a (Trái)` theo hướng nhìn của chính robot; màn hình và slide dùng hướng nhìn theo cam NGANG, nên ngược lại.

Từ vật tới phím (nhóm 1):

1. Đổi vị trí vật từ cm trên giấy sang mm trong hệ robot, theo vị trí và hướng của đế.
2. Đế quay θ_1 = atan2(y, x); vai và khuỷu giải tam giác hai khâu, điểm kẹp cao 25 mm (nửa thân cốc). Không có servo cổ tay nên không chọn được hướng ngón kẹp.
3. Đổi góc khớp sang góc servo, làm tròn về bội số của 2°.
4. Mỗi servo thành một `Move`: phím, số lần (= độ lệch ÷ bước), hướng, số độ. Kẹp không bao giờ nằm trong `Plan`.

Ví dụ với `calibration.json` hiện tại: cốc ở (20,1; 7,4) cm → gợi ý đế 74°, vai 58°, khuỷu 90°. Từ Home: `D` × 8 và `S` × 16; khuỷu đã đúng.

Mô hình cánh tay đã hiệu chỉnh: mô hình mặc định (số đo thước) lệch 3,5–5,9 cm. Khớp lại từ 10 lần "ôm" cốc bằng tay cho sai số RMS 8,5 mm (bỏ-một-mẫu 11,6 mm). Kết quả chỉ đúng với tờ giấy và vị trí robot lúc đo; đổi một trong hai thì phải đo lại.

Firmware gửi JSON `{"t":…,"servo":[đế,vai,khuỷu,kẹp],"relax":0|1,"armrelax":0|1}` qua USB. Góc này là **góc lệnh** firmware đã gán; MG996R không báo vị trí thật.

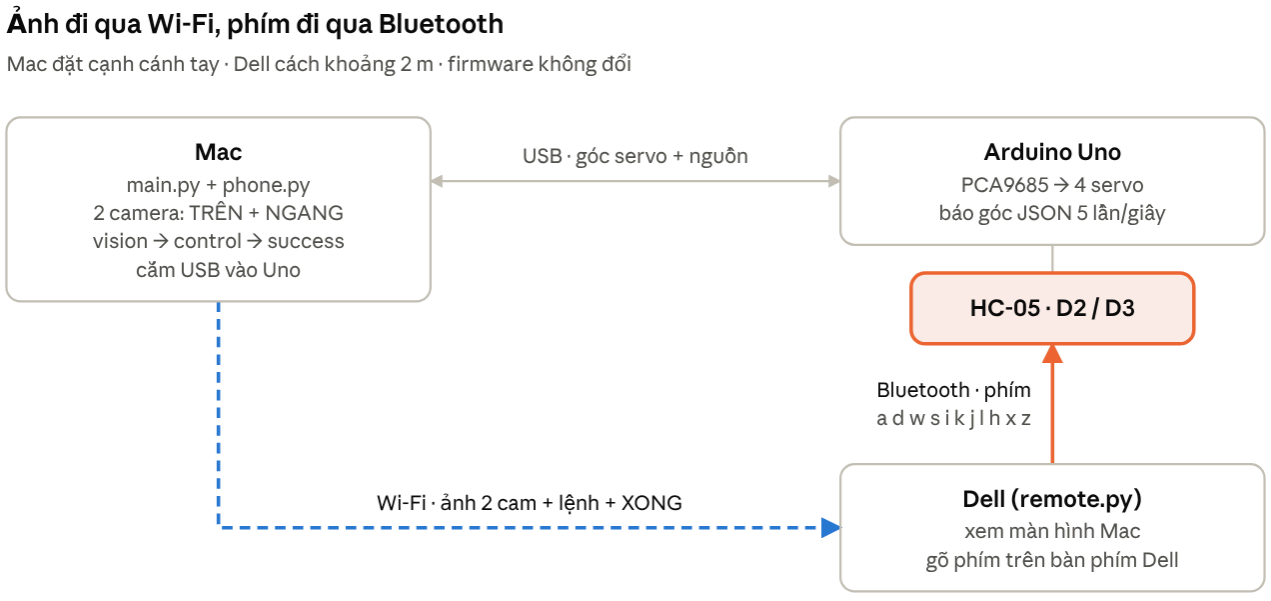
Bảo vệ servo (chống nóng):

- Giới hạn kẹp 160° (xung khoảng 2,56 ms); bản cũ 175° vượt dải servo và làm servo nóng.
- Kẹp mở hết cỡ (95°) 5 giây không có lệnh → ngắt xung; đã khép quá 60 giây → ngắt xung.
- Chỉ phím di chuyển mới cấp xung lại. Kẹp bị ngắt xung thì tự sập hẹp lại; `Z` làm cánh tay sụp theo trọng lực, nên hạ đầu kẹp xuống chỗ trống trước.
- Uno tự reset khi một chương trình khác mở rồi đóng cổng USB lúc `main.py` đang chạy. `main.py` nhận ra (thời gian `t` quay về 0) và báo `ARDUINO VUA RESET`.

Firmware cuối (phím Z, bước 2° / 5°, giới hạn 160°) đã nạp và kiểm trên mạch thật ngày 02/11; phần điều khiển từ xa không cần sửa firmware.

6. Điều khiển từ xa (Mac + Dell)

Máy Dell cách khoảng 2 m xem màn hình Mac qua **Wi-Fi** và gửi phím qua **Bluetooth** tới HC-05 trên Uno; firmware không đổi.



điều khiển từ xa · 3 máy, 3 đường nối

Mỗi đường chở một thứ: USB mang góc servo về Mac, Wi-Fi mang ảnh màn hình sang Dell, Bluetooth mang phím từ Dell tới HC-05.

- **Mac** (`main.py` + `phone.py`): giữ 2 camera, cắm USB vào Arduino. Chia sẻ ảnh màn hình (2 cam + lệnh + XONG) qua một máy chủ web nhỏ, tối đa 10 ảnh/giây.
- **Đường dẫn có mã 6 số**: `http://<ip Mac>:8000/<mã>/`. Mã lưu trong `.phone_pin` (không đưa lên git), giữ nguyên giữa các lần chạy; đổi mạng thì chỉ IP đổi. Sai mã → 404.
- **Dell** (`remote.py`, cần `opencv-python`, `numpy`, `pyserial`): một cửa sổ hiện màn hình Mac, thanh trạng thái trên cùng báo Wi-Fi và Bluetooth. Gõ `A D W S I K J L H X Z` thì phím đi qua Bluetooth; `Q` thoát.
- **An toàn**: `H` (Home) và `Z` (nghỉ hết) phải bấm 2 lần trong 2 giây. Ảnh cũ quá 3 giây thì viền đỏ.
- **Tự tìm cổng**: chỉ chọn cổng Bluetooth giống HC-05 (cổng gọi đi kiểu `LOCALMFG`), bỏ cổng của điện thoại, laptop và cổng chiều vào; thử lần lượt nếu có nhiều cổng.

Vì sao ảnh không đi qua Bluetooth: HC-05 ở 9600 baud chỉ được khoảng 1 KB/giây, còn một ảnh màn hình JPEG khoảng 120 KB, tức khoảng 2 phút mỗi ảnh. Phím chỉ 1 byte nên Bluetooth đủ.

Các bước chạy:

1. Dell: ghép đôi HC-05 (PIN thường là `1234` hoặc `0000`). Windows tạo 2 cổng COM; dùng cổng **Outgoing**. Kiểm bằng `python remote.py --list-ports`.
2. Mac và Dell vào cùng một Wi-Fi. Mạng trường chặn hai máy thấy nhau thì cả hai bắt điểm phát của điện thoại.
3. Mac: cắm USB Arduino, chạy `uv run python main.py` từ Terminal.app, chép dòng `python remote.py --mac http://…` nó in ra.
4. Dell: chạy đúng dòng đó (thêm `--port COMx` nếu tự tìm sai cổng).

"Not connected" trong cài đặt Bluetooth của Windows là bình thường: HC-05 chỉ nối khi có app mở cổng COM. HC-05 nhận mỗi lúc một kết nối, nên chỉ ghép đôi với Dell.

Dự phòng: Dell mở đường dẫn bằng trình duyệt để xem và gửi phím bằng PuTTY (Serial, cổng HC-05, 9600). Điện thoại mở đường dẫn (hoặc quét mã QR in trong Terminal) cũng xem được và có nút bấm; nút này đi qua Wi-Fi tới Mac rồi USB. Khác mạng: `ngrok http 8000`.

7. Kiểm thử và số liệu

124 test pass trong khoảng 11 giây (chạy lại ngày 05/11), chia trong 13 tệp; nhiều nhất là `test_control` (29) và `test_vision` (15).

- Không test nào chạm camera, cổng Serial hay `calibration.json` thật: cảnh dựng bằng camera giả lập có đáp án chính xác.
- IK ↔ FK: trên lưới hơn 100 điểm trong tầm với, FK(IK(p)) trùng p với sai số dưới 0,1 mm.

- Chuỗi nối nhau (`test_end_to_end`): bấm đúng lệnh trên màn hình thì ra XONG, đếm đúng 1 lần; bấm lệch 4° thì hết XONG nhưng số đã đếm không mất.
- Điều khiển từ xa (`test_phone` 6, `test_remote` 9): chọn đúng cổng HC-05 trong danh sách cổng thật của Dell, H / Z phải bấm 2 lần, mất kết nối thì báo, sai mã thì 404.

Chạy lại:

```
uv run pytest
.venv/bin/python -B slides/tools/check_facts.py   # so số liệu slide với mã hiện tại
```

Chạy cả chương trình không cần phần cứng (dùng ảnh chụp bằng phím `C`):

```
uv run python main.py --top captures/tren.jpg --side captures/ngang.jpg --no-serial --headless --seconds 10 --dump-dir out/
```

Số liệu slide (`slides/05_SO_LIEU_VA_NGUON.md`) và slide Kết quả đã cập nhật theo các số này ngày 05/11: 124 test, 13 tệp test; `check_facts.py --deep` báo KHỚP.

8. Slide thuyết trình

Bộ slide đã dựng xong: `slides/Project_IoT_ThuyetTrinh.pptx` (+ `.pdf`), **26 slide**, màu slide Gamota, chia 4 phần cho 4 người; mỗi phần mở bằng một slide ngăn.

Slide	Phần	Nội dung
1–4	Mở đầu	bìa, thành viên, bài toán, sơ đồ khối hệ thống
5–10	01 · Linh kiện	danh sách linh kiện, Arduino Uno R3 và PCA9685, servo / nguồn / HC-05 / camera, sơ đồ mạch, bảo vệ servo kẹp
11–13	02 · Computer vision	hệ tọa độ từ tờ A4, nhận diện vật
14–20	03 · Code điều khiển	lưu đồ thuật toán, code firmware, động học ngược, giao diện, điều khiển từ xa, hiệu chỉnh mô hình
21–25	04 · Success	điều kiện hoàn thành, thử nghiệm trên cánh tay thật, kết quả, hạn chế và hướng phát triển
26	Kết	tổng kết

- Chữ trên slide ngắn, từ ngữ trung tính; giải thích và câu "Nếu được hỏi" nằm trong ghi chú của từng slide.

- Mọi con số lấy từ `slides/05_SO_LIEU_VA_NGUON.md`, kèm nhãn [CODE], [ĐO], [MÔ PHỎNG], [ƯỚC LƯỢNG], [CHƯA]. Mục 14 của tệp đó liệt kê các câu **cấm nói**, ví dụ "robot tự động gắp" hay bất kỳ tỉ lệ thành công nào.
- Dựng lại: `build_slides.py` → `export_pdf.py` → `render_previews.py`; kiểm bằng `verify_slides.py` và `tools/check_facts.py`.

Việc còn lại trước khi thuyết trình:

- [ ] Điền tên 4 người vào bảng `TEAM` đầu `build_slides.py` (đang là `[Tên A]` … `[Tên D]`), rồi dựng lại
- [x] ~~Đổi số test trên slide Kết quả sang 124 và số tệp test sang 13 (đã làm 05/11)~~
- [ ] Điền câu trả lời về cầu chia áp chân RXD của HC-05 trong ghi chú slide Sơ đồ mạch
- [ ] Xác nhận "nhóm 1 / nhóm 2" là hai phần mã hay hai nhóm người
- [ ] Nếu chạy được trên cánh tay thật trước buổi thuyết trình: cập nhật các dòng "chưa" ở slide Điều khiển từ xa và Kết quả

9. Cách chạy và xử lý sự cố

Trên Mac: cài một lần bằng `uv sync`, rồi chạy `uv run python main.py` từ **Terminal.app** (Terminal cần quyền Camera của macOS).

```
uv sync                                  # cài OpenCV, pyserial, pytest...
./robot_arm_4dof/flash.sh                # nạp firmware (chỉ khi sửa firmware)
uv run python main.py                    # chạy chính
uv run python main.py --list-cameras     # xem camera nào được gán vai trò nào
uv run python main.py --no-serial        # mô phỏng, không mở cổng Arduino
uv run python main.py --no-phone         # tắt chia sẻ màn hình cho Dell
```

Hiện tượng	Cách xử lý
Camera đen, log `not authorized to capture video`	chạy từ Terminal.app và cấp quyền Camera trong Cài đặt hệ thống → Quyền riêng tư
Không thấy OBSBOT	cắm lại OBSBOT; kiểm bằng `--list-cameras`; sai vai trò thì bấm `T`
`KHONG THAY TO A4`	in đúng kích thước, đủ sáng, trong khung có ít nhất 2 mã không bị che
Lưới lệch sau khi dời camera / giấy	bấm `U`

Hiện tượng	Cách xử lý
`CHUA THAY VAT`	đặt vật có màu (cốc đỏ), không sát mép giấy; `TAB` đổi vật
`USB Arduino: MO PHONG`	cắm cáp USB, tắt chương trình khác đang giữ cổng
Cánh tay tự giật về Home	Uno bị reset do chương trình khác mở cổng USB; đừng mở cổng khi `main.py` đang chạy
Gợi ý đưa kẹp lệch vài cm	đo lại hiệu chỉnh; kiểm tờ giấy không bị xê dịch
Dell: `khong thay Mac`	hai máy cùng Wi-Fi chưa, `main.py` đang chạy chưa; mạng trường chặn thì dùng điểm phát điện thoại
Dell: `sai ma trong duong dan`	chép lại dòng `python remote.py --mac …` mới nhất `main.py` in ra (mã giữ nguyên, IP có thể đổi)
Dell: `chua thay HC-05` / `khong noi duoc COMx`	ghép đôi HC-05; HC-05 có điện (đèn nháy); ngắt thiết bị khác đang nối HC-05; thử `--port` cổng Outgoing
Chạy `remote.py` trên Mac báo lỗi	`remote.py` chạy trên **Dell**, không chạy trên Mac

10. Giới hạn và việc chưa xác minh

Chưa có lần đếm thành công nào trên cánh tay thật với bản hiện tại; phần lớn kết quả đến từ test, camera giả lập và ảnh đã lưu.

Việc	Đã kiểm trên phần cứng thật?	Ghi chú
Firmware cuối (phím Z, 2° / 5°, kẹp 160°)	có, 02/11	banner `+ Z (nghi het)`, JSON có `armrelax`
Kẹp và chuyển cốc bằng bàn phím	có, 02/11 20:32–20:36	với màn hình bản cũ; cốc đổ khoảng 3 lần
Khớp mô hình cánh tay	có, 10 lần ôm cốc	RMS 8,5 mm trên chính 10 mẫu đó (bỏ-một-mẫu 11,6 mm), không phải kiểm độc lập

Việc	Đã kiểm trên phần cứng thật?	Ghi chú
Bấm theo gợi ý của bản mới rồi xem hai ngón có bao quanh cốc không	chưa	
Đếm thành công (`Thanh cong: N lan`)	chưa	chỉ có trong test và mô phỏng
Tỉ lệ thành công	chưa đo	kế hoạch 28/10 đặt mục tiêu ≥ 80% trên 30 lần
Nhiệt độ servo sau khi thêm ngắt xung	chưa đo	
Dell nối Bluetooth tới HC-05	có, 04/11	thanh trạng thái báo `COM12 da noi`
Dell bấm phím → cánh tay quay → Mac cập nhật góc → Dell thấy	chưa	vòng đầy đủ mới chạy với cánh tay mô phỏng

Giới hạn của thiết kế:

- **XONG là góc lệnh**: XONG chứng minh đã bấm đúng số lần, không chứng minh hai ngón kẹp thật đang ở sát vật (MG996R không báo vị trí).
- **Kẹp không giữ chắc cốc thon**: ở 150–160° cốc bị đẩy ra "như hạt dưa", nên đề tài chỉ xét 3 góc servo. Cách chữa đề xuất: dán ma sát lên mặt trong hai ngón hoặc dùng vật thân thẳng.
- **Hiệu chỉnh găn với tờ giấy cũ** (in nhỏ hơn đúng cỡ khoảng 4,5%) và vị trí robot lúc đo; đổi tờ giấy hoặc dời robot thì phải đo lại.
- **Không có servo cổ tay**: không chọn được hướng tiếp cận của ngón kẹp.
- **So với kế hoạch 28/10**: không có ESP32, MQTT, dashboard, cảm biến lực FSR hay 6 servo; chỉ nhắc chúng như "điều chỉnh so với kế hoạch".

Nhánh `feature/remote-control` (phần điều khiển từ xa) đã đẩy lên GitHub nhưng chưa gộp vào `main`.